SQL QUERY

*JOIN

-Types of JOIN

1)Inner Join:-

i)Equi Join

➔To perform equi join between the two tables there must be a common field need to be same and fk-pf relationship is not compulsory.

➔equi join is performed on the common field with same datatype.

➔**select*from student INNER JOIN course ON student.student_id =course.student_id;**

STUDENT_ID	NAME
101	Lalit
102	Gaurav
103	Sarvesh

STUDENT_ID	COURSE
102	english
105	math
103	science
107	computer

select name,course from student INNER JOIN course ON student.student_id=course.student_id;

NAME	COURSE
Gaurav	english
Sarvesh	science

ii)Non-Equi Join

➔Non-Equi join is performed between the tables not sharing common field.

➔here join condition is not based on “=” operator it is based on > < between operators.

➔**SELECT*FROM Student,Course WHERE Student.STUDENT_ID > Course.Student_ID;**

STUDENT_ID	NAME
101	Lalit
102	Gaurav
103	Sarvesh

STUDENT_ID	COURSE
102	english
105	math
103	science
107	computer

➔

STUDENT_ID	NAME	STUDENT_ID	COURSE
103	Sarvesh	102	english

2)Outer Join

➔Inner joins returns only matching rows but will not return unmatched rows, to display unmatched rows perform outer join.

i)Left Outer Join:-

➔Returns all rows(matched + unmatched) from left side table and matching rows form right side table.

STUDENT_ID	NAME
101	Lalit
102	Gaurav
103	Sarvesh

STUDENT_ID	COURSE
102	english
105	math
103	science
107	computer

➔**SELECT * FROM STUDENT S LEFT OUTER JOIN COURSE C ON S.STUDENT_ID = C.STUDENT_ID;**

➔

STUDENT_ID	NAME	STUDENT_ID	COURSE
102	Gaurav	102	english
103	Sarvesh	103	science
101	Lalit		

ii)Rigth Outer Join:-

➔Returns all rows(matched+unmatched) from right side table and

matched rows from left side table.

STUDENT_ID	NAME
101	Lalit
102	Gaurav
103	Sarvesh

STUDENT_ID	COURSE
102	english
105	math
103	science
107	computer

➔**SELECT * FROM STUDENT S RIGHT OUTER JOIN COURSE C ON S.STUDENT_ID = C.STUDENT_ID;**

➔

STUDENT_ID	NAME	STUDENT_ID	COURSE
102	Gaurav	102	english
103	Sarvesh	103	science
		107	computer
		105	math

iii)Full Outer Join:-

➔Returns all rows(metched+unmatched) from both tables.

STUDENT_ID	NAME
101	Lalit
102	Gaurav
103	Sarvesh

STUDENT_ID	COURSE
102	english
105	math
103	science
107	computer

➔**SELECT * FROM STUDENT S FULL OUTER JOIN COURSE C ON S.STUDENT_ID = C.STUDENT_ID;**

➔

STUDENT_ID	NAME	STUDENT_ID	COURSE
102	Gaurav	102	english
		105	math
103	Sarvesh	103	science
		107	computer
101	Lalit		

3)Cross Join

➔Cross join return product or cartesion product of two tables.

STUDENT_ID	NAME
101	Lalit
102	Gaurav
103	Sarvesh

STUDENT_ID	COURSE
102	english
105	math
103	science
107	computer

➔**SELECT S.NAME,C.COURSE FROM STUDENT S CROSS JOIN COURSE C;**

STUDENT_ID	NAME	STUDENT_ID	COURSE
101	Lalit	102	english
101	Lalit	105	math
101	Lalit	103	science
101	Lalit	107	computer
102	Gaurav	102	english
102	Gaurav	105	math
102	Gaurav	103	science
102	Gaurav	107	computer
103	Sarvesh	102	english
103	Sarvesh	105	math
103	Sarvesh	103	science

STUDENT_ID	NAME	STUDENT_ID	COURSE
103	Sarvesh	107	computer

12 rows selected.

-Ranking Function

➔To assign the rank to each row wise & group of row wise.

i)Row_Number()

ii)Rank()

iii)Dense_Rank()

➔**Syntax:-**

RANKING FUNCTION() OVER(PARTITION BY <COLUMN NAME>

ORDER BY <COLUMN NAME> [ASC / DESC])

*Here

➔PARTITION BY------→ OPTIONAL (ON GROUP OF ROWS)

➔ORDER BY ---------→ MANDATORY CLAUSE

i)Row_Number()

➔ROW_NUMBER() is a ranking function in SQL that gives a unique number to each row in your result, starting from 1, based on the order you choose.

EMPNO	ENAME	SAL	DEPTNO
1	A	3000	10
2	B	4000	20
3	C	5000	30
4	D	6000	20
5	E	5000	10

➔SELECT ENAME,SAL,ROW_NUMBER() OVER(ORDER BY SAL DESC) AS RANK FROM EMP;

ENAME	SAL	RANK
D	6000	1
C	5000	2
E	5000	3
B	4000	4
A	3000	5

ii)Rank()

➔This means, if two rows have the same value, they will both get the same rank, but the next row will get a rank skipped by the number of ties.

EMPNO	ENAME	SAL	DEPTNO
1	A	3000	10
2	B	4000	20
3	C	5000	30
4	D	6000	20
5	E	5000	10

➔SELECT ENAME,SAL,RANK() OVER(ORDER BY SAL DESC)AS RANK FROM EMP;

ENAME	SAL	RANK
D	6000	1
C	5000	2
E	5000	2
B	4000	4
A	3000	5

iii)Dense_Rank

➔if two rows have the same value, they get the same rank, but the next rank continues without skipping.

EMPNO	ENAME	SAL	DEPTNO
1	A	3000	10
2	B	4000	20
3	C	5000	30
4	D	6000	20
5	E	5000	10

➔SELECT ENAME,SAL,DENSE_RANK() OVER(ORDER BY SAL DESC)AS RANK FROM EMP;

ENAME	SAL	RANK
D	6000	1
C	5000	2
E	5000	2
B	4000	3
A	3000	4

View

➔A view in SQL is like a virtual table. It doesn't store data itself — instead, it shows data from one or more real tables based on a query you write.

1)Simple view

2)Complex view

3)Read only view

4)with check option

5)force view

i)Simple view

➔The view which contain a sub-query that retrieve from one base table or view is called simple view.

-Syntax:-

CREATE VIEW VIEW NAME AS SELECT * FROM TABLE-NAME;

EMPNO	ENAME	SAL	DEPTNO
1	A	3000	10
2	B	4000	20
3	C	5000	30
4	D	6000	20
5	E	5000	10
6	F	3000	50
5	G	5000	30

➔ CREATE VIEW EMP_VIEW1 AS SELECT * FROM EMP;

EMPNO	ENAME	SAL	DEPTNO
1	A	3000	10
2	B	4000	20
3	C	5000	30
4	D	6000	20
5	E	5000	10
6	F	3000	50
5	G	5000	30

➔CREATE OR REPLACE VIEW EMP_VIEW2 AS SELECT * FROM EMP WHERE DEPTNO=20;

EMPNO	ENAME	SAL	DEPTNO
2	B	4000	20
4	D	6000	20

ii)Complex view:-

➔Complex view contain sub-queries where sub-queries contain query.

➔The view which is constructed with the help of more then one-base Table then its called Complex view.

➔Syntax:-

CREATE VIEW VIEW-NAME AS SELECT*FROM TABLE-NAME1 UNION SELECT*FROM TABLE-NAME2;

EMPNO	ENAME	SAL
1	A	3000
2	B	4000
3	C	5000
4	D	6000
5	E	5000
6	F	3000
5	G	5000

DEPT_EID	DNAME	DEPTEMPNO
10	SALE	100
20	HR	200
20	ACCOUNT	300
40	DEVELOPER	400

➔ CREATE VIEW VNAME AS SELECT*FROM EMP UNION SELECT*FROM DEPT1;

➔

EMPNO	ENAME	SAL
1	A	3000
2	B	4000
3	C	5000
4	D	6000
5	E	5000
6	F	3000
5	G	5000
10	SALE	100
20	HR	200
20	ACCOUNT	300
40	DEVELOPER	400

iii)Read only view:-

➔It only allow to perfrom read operation on base table , It restricts to perform write operation.

➔Syntax:-

CREATE VIEW VIEW-NAME AS SELECT * FROM TABLE-NAME WITH READ ONLY;

EMPNO	ENAME	SAL
1	A	3000
2	B	4000
3	C	5000
4	D	6000
5	E	5000
6	F	3000
5	G	5000

➔CREATE VIEW RVIEW AS SELECT * FROM EMP WITH READ ONLY;

➔

EMPNO	ENAME	SAL
1	A	3000
2	B	4000
3	C	5000
4	D	6000
5	E	5000
6	F	3000
5	G	5000

iv)With check option:-

➔In with check option view if the given condition satisfy in selected column then allow to perform write operation otherwise restricted.

➔**Syntax:-**

CREATE VIEW VIEW-NAME AS SELECT*FROM TABLE-NAME WHERE CONDITION WITH CHECK OPTION;

v)Force view:-

➔Force view is used basically for the situation when we create a view using a table but the table is not exists , at that time we use force view.

➔**Syntax:-**

CREATE FORCE VIEW VIEW-NAME AS SELECT * FROM TABLE-NAME;

➔ **CREATE FORCE VIEW FVIEW AS SELECT * FROM STU1;**

Note(View created with compilation error)

Sub Queries/Nested Queries

➔The requirement where we don't reach by query those kind of operation we perform sub queries/nested sub queries.

➔A query inside another query is called nested sub query statement.

➔**Syntax:- outer query** **Inner query**

SELECT*FROM <TN> WHERE <CONDITION>
(SELECT*FROM……(SELECT*FROM……(SELECT*FROM……)));

➔As per the execution process of sub-queries it again classified in two ways.

i)Non-Corelated sub query:-

➔In NCRSQ -➔First Inner Query Is Executed And Later Outer Query Will Execute.

➔i.e Outer query is always depends on the result of inner query.

ii)Corelated sub query:-

➔In CRSQ--→ First Outer Query Is Executed And Later Inner Query Will Execute.

➔i.e Inner query is always depends on result of outer query.

Working with non-corelated sub queries

Def:- First Inner Query Is Executed And Later Outer Query Will Execute.

➔Types of non-corelated sub queries

i)Single-row sub query

➔ When a sub query returns a single value is called as simple/single row subquery.

Ex

1)WAQ to get employee details from the table who are getting the first highest salary in the table?

➔Step1:-

Inner Query:- SELECT MAX(SAL) FROM EMP;--→Return value--→ 45000

Step2:-

Outer Query:-SELECT*FROM EMP WHERE SALARY=(INNER QUERY);

Step3:-

SUBQUERY=OUTER+INNER

====➔SELECT*FROM EMP WHERE SAL=(SELECT MAX(SAL) FROM EMP;

ii)Multiple-Row Subquery

➔A subquery returns more than one value is called as multiple row sub-query. We can use operator is “IN”.

Ex:-

2)WAQ to display employee details from the table whose department is same as the department of Lalit and Gaurav?

➔Step1:

Inner Query:-SELECT DEPT FROM EMP WHERE ENAME=’LALIT’ OR ‘GAURAV’ FROM EMP;

Step2:

Outer Query:-SELECT*FROM EMP WHERE DEPT IN(INNERQUERY);

==➔ **SELECT*FROM EMP WHERE DEPT IN(SELECT DEPT FROM EMP WHERE ENAME='LALIT' OR ENAME='GAURAV');**

PL-SQL

-What is PL/SQL? Explaination?

➔PL/SQL(Procedural language extension of SQL is a powerful database programming language specially designed for oracle database)

➔-Syntax:-

```
BEGIN
        DBMS_OUTPUT.PUT_LINE('Hello World');
END;
```

-Note

i)PL/SQL programs are compiled by the oracle database server and stored inside the database.

ii)PL/SQL inherits the security & portability of the oracle database.

-Why PL/SQL has been introduce:-

➔Pl/SQL was developed to address the limitation of SQL.

→**SET SERVEROUTPUT ON**(To start server)

***Program to store value in variables:-**

declare

<declaration section>

begin

<executable commands>

exception (optional)

<handle exception>

end;

/

➔

```
DECLARE
      NAME VARCHAR2(12);
      AGE NUMBER;
BEGIN
     NAME:='LALIT';
     AGE:=24;

     DBMS_OUTPUT.PUT_LINE('MYNAME:'||NAME);
     DBMS_OUTPUT.PUT_LINE('AGE:'||AGE);
END;
/
```

➔

```
MYNAME:LALIT
AGE:24
```

➔CRUD Operation using PL/SQL

-Create table:-

```
BEGIN
EXECUTE IMMEDIATE 'CREATE TABLE STUD(SID NUMBER,SNAME VARCHAR2(10))';
DBMS_OUTPUT.PUT_LINE('TABLE CREATED SUCCESSFULLY:)');
END;
/
```

-Function in PL/SQL

-What is function?full explaination.

➔Function is a block of code that perform a given task and return value.

Note:-i)It only runs when it call.

ii)It saves our lots of efforts.

Types of Function

i)Scalar function

ii)Table function

-Syntax

CREATE OR REPLACE FUNCTION FUN-NAME(PARA)

RETURN NUMBER IS

RESULT NUMBER;

BEGIN

//CODE

END;

-Triggers

➔A trigger is a course of action to be taken when any DDl,DML event occurs.

➔It is named PL/SQL block like a procedure.

➔But it is executed implicitly by oracle server.

➔**Types of Triggers**

i)DML Triggers(Table Level)

ii)DDL Triggers(Database Level)

➔**Triggers are created to**

1. Control DML
2. Implement complex business rule
3. Implement complex validation
4. Generate values for primary key
5. For auditing
6. Maintain replicas

➔**General Syntax for Triggers**

```
Create [or replace] trigger trigger_name
[before|after|instead of]
[insert [or] |update [or] | delete] on table_name
[referencing old as 0 new as n]
[for each row][when (condition)]
[declare <declare section>]
Begin
[exception]
End;
```

-Befor Trigger

➔Before trigger is executed before executing a DML command.

➔Sequence:

Triggers execution

DML Execution

➔DML Triggers

These triggers are executed automatically whenever user performs DML operation on the table

➔Levels of DML Triggers

i)Statement level(default):- executed only once for the statement.

ii)Row level:-executed once for each row affected by the DML.